LipidIN × Agent — `code_for_PB` User Manual (English)

> Paternò–Büchi (PB) reaction pipeline for **carbon–carbon double-bond position localization** in lipids, driven either as a one-shot batch script or through a conversational web agent.
>
> A Chinese version of this manual is in [`README_ZH.md`](README_ZH.md) .

This manual is written from an actual read of the whole pipeline and a real run of the bundled demo. All screenshots below are from the live interface; the figures are produced by the demo data.

---

1. What this does

Conventional lipidomics tells you *how much* of a lipid species (e.g. `PC 42:8`) is present but not *where* its double bonds sit. PB derivatization tags each C=C double bond so it can be pinpointed. This pipeline:

1. **Stage 1 — Conventional annotation** of the underivatized sample (`common/` , negative mode, acetate library) → identifies species and quantifies them.
2. **Stage 2 — PB search** of the derivatized sample (`PB/` , positive `[M+H]⁺` , PB library) → finds the diagnostic ion pairs.
3. **Stage 3 — Double-bond localization** → reconstructs positions in omega notation (`n-9` , `n-6` , ...).
4. **Stage 4 — Chain × double-bond-position quantification** with total-area normalization.

The web agent adds downstream steps: experiment design, QC + differential analysis (species layer **and** double-bond-position layer), publication figures, LLM interpretation, hypotheses (literature + LIPID MAPS), verification, and a Markdown report — **11 stages** in total.

---

2. Requirements

Item	Requirement
OS / Python	**Windows x64 + CPython 3.11** (the C++ engines ship as `*.cp311-win_amd64.pyd`)
RAM	**≥ 16 GB** (the theoretical libraries are large)
Python packages	`flask pandas numpy scipy scikit-learn matplotlib polars plotly pybind11 fisher_py`
Thermo `.raw` reading	`fisher_py` + .NET runtime (bundled with fisher_py)
Spectral libraries	**Download from Zenodo → https://zenodo.org/records/21421866** (`PB_PX_H.pkl` , `common_PX_CH3COO.pkl` , `common_PX_CH3COO.pkl.ms1index.pkl`)
LLM (optional)	An API key for Anthropic / OpenAI-compatible / DeepSeek. Without it, structured commands still work; only free-form design/interpretation/hypotheses need it.

Put the three library files into a folder (e.g. `PB library/`). Note: `fisher_py` reads Thermo `.raw` directly — only **strict DDA** acquisitions are accepted (DIA/SWATH/PASEF are rejected on purpose).

---

3. Prepare your data

The pipeline needs **two acquisitions of the same samples**, in two subfolders:

```
<raw root>/
 ├─ common/        # underivatized, negative mode  → MSMS1.raw, MSMS2.raw,  …
 └─ PB/            # PB-derivatized, positive mode  → PB1_A1.raw, PB2_A1.raw,  …
```

- `PB1_*` and `PB2_*` are **two technical injections of the same sample** — the pipeline merges them automatically in Stage 4.
- The bundled demo lives at `D:\bio_inf\LipidIN_github\PB_demo\demo` (with `common/` and `PB/`).

---

4. Two ways to run

Option A — One-shot batch (`run.py`)

Open `run.py` and edit the path block near the top (`PROJECT_ROOT` / `SAMPLE_ROOT` / `LIBRARY_DIR`, or point `RAW_ROOT` at a folder containing `common/` and `PB/`). Then:

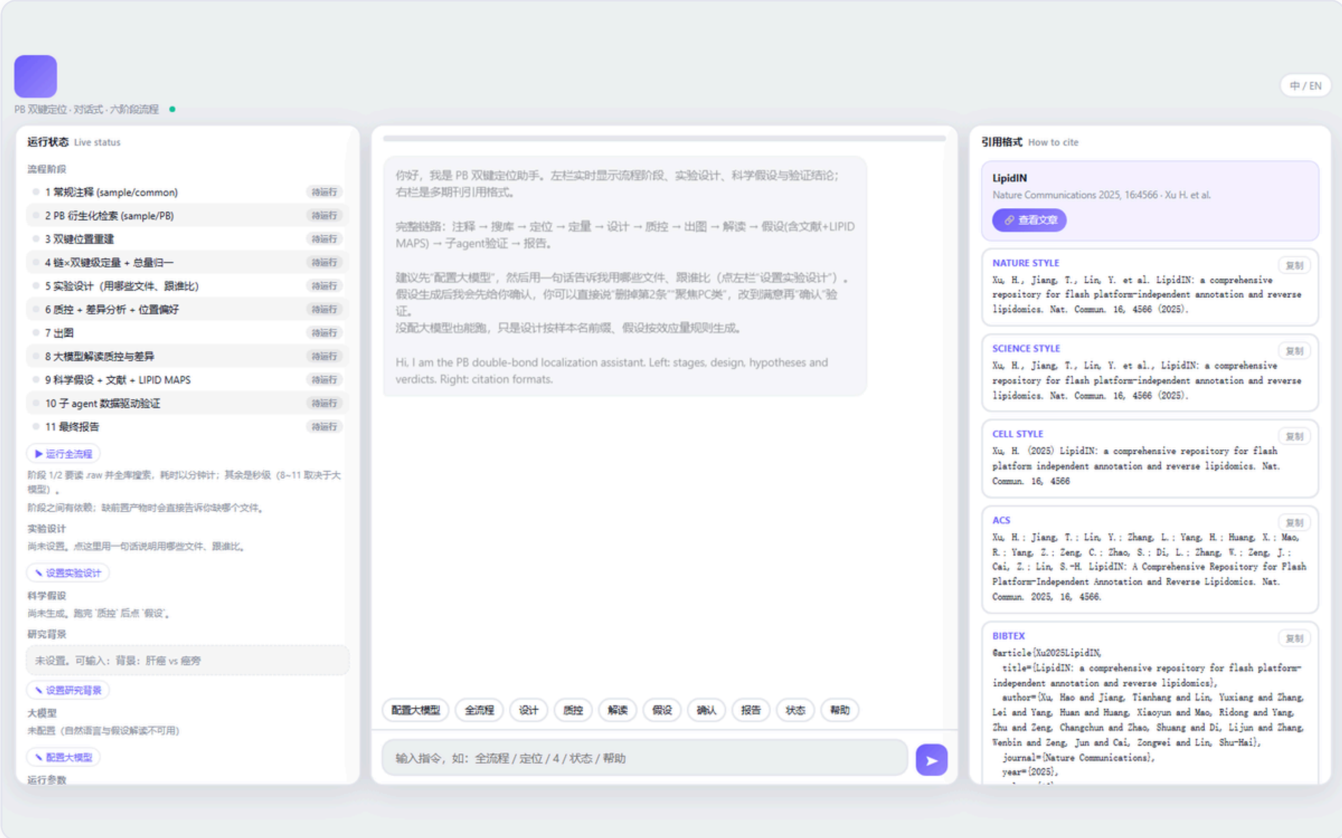
```
D:\anaconda3\python.exe run.py
```

It runs Stage 1 → 4 in order and writes all CSV outputs into each `.../pkl/` folder. Toggle `RUN_STAGE1..4` at the top to run only part of the pipeline. Keep `EQ3_WORKERS = 1` on a 16 GB machine.

Option B — Conversational agent (recommended)

```
D:\anaconda3\python.exe pb_agent_server.py --no-open --port 8790
# then open http://127.0.0.1:8790 in your browser
```

You get a three-pane interface: **left** = live stage status + parameters, **center** = chat, **right** = citation formats.

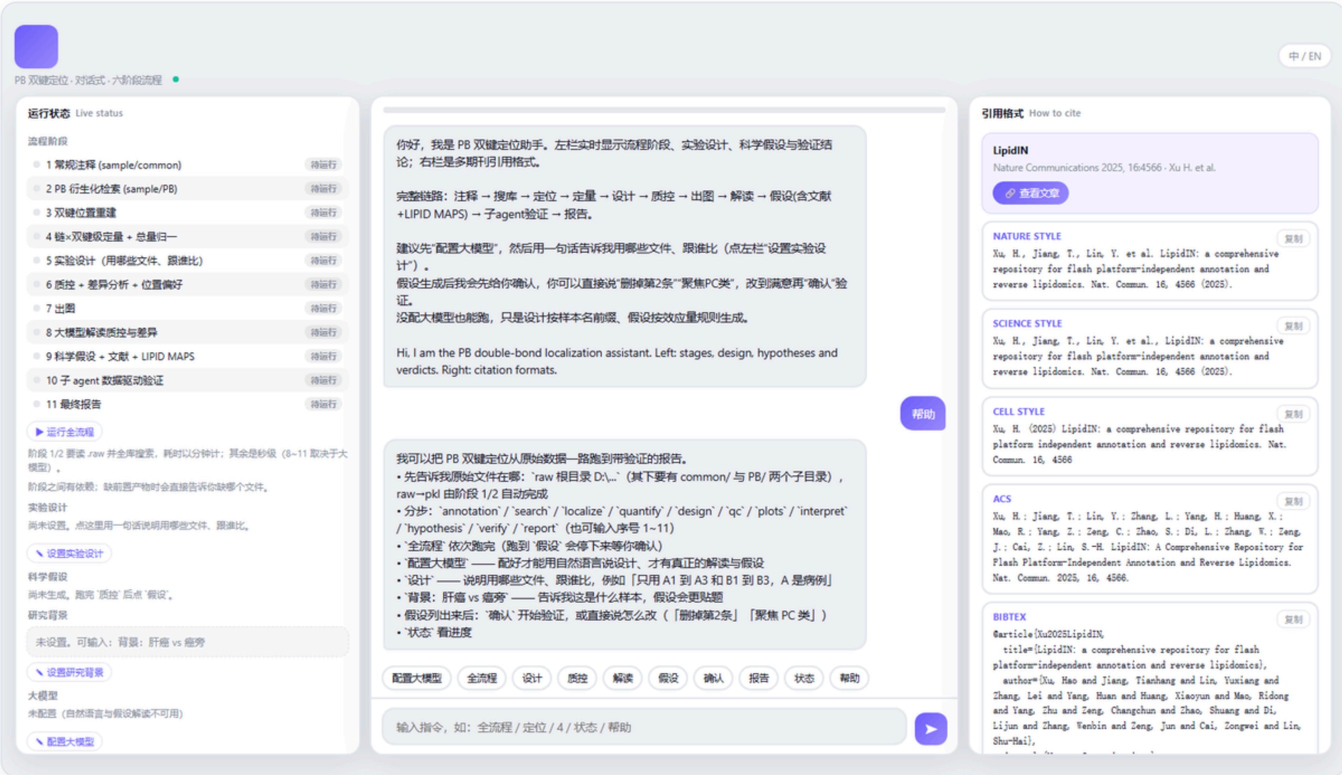


5. A full standard conversation

Below is a real end-to-end session against the demo. Type these into the chat box (or click the chips).

5.1 Ask for help — `帮助` / `help`

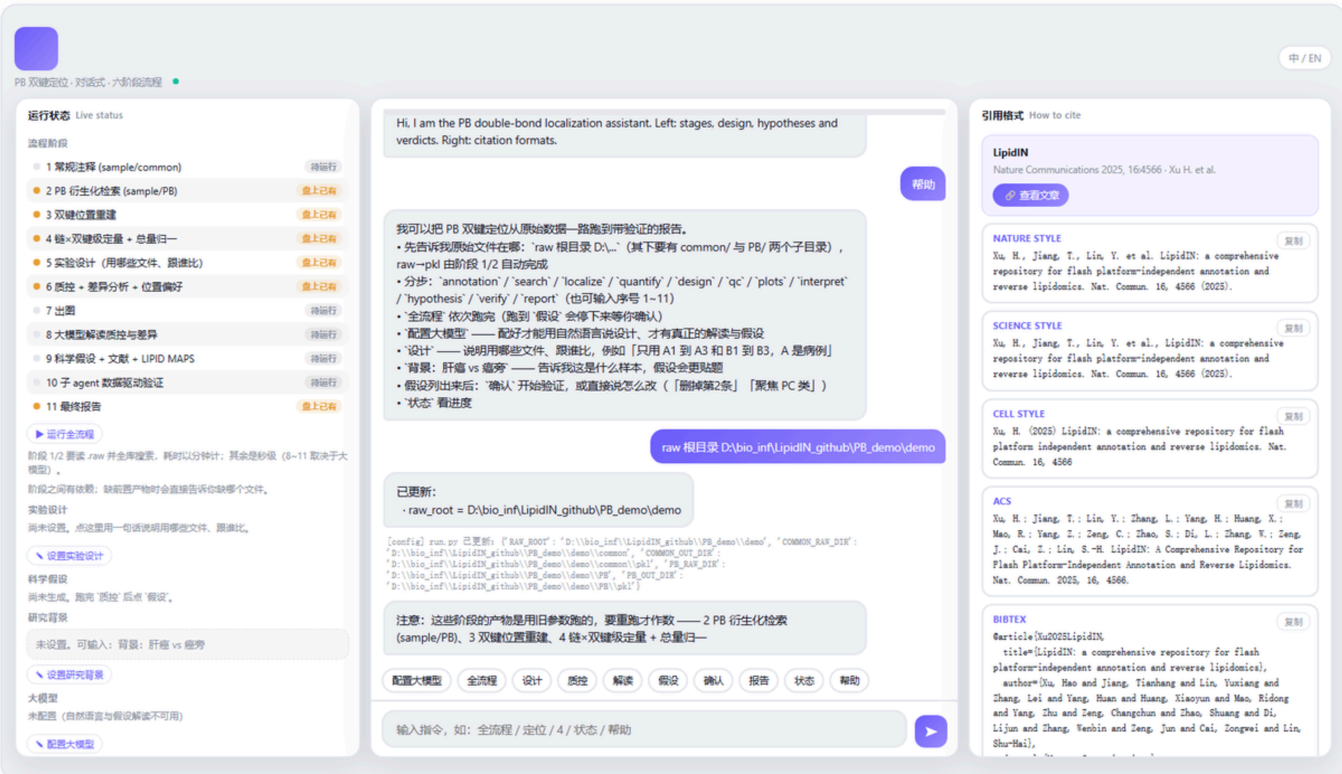
The agent lists every command and the full path `annotation → search → localize → quantify → design → qc → plots → interpret → hypothesis → verify → report` (you can also type the stage number `1` – `11`).



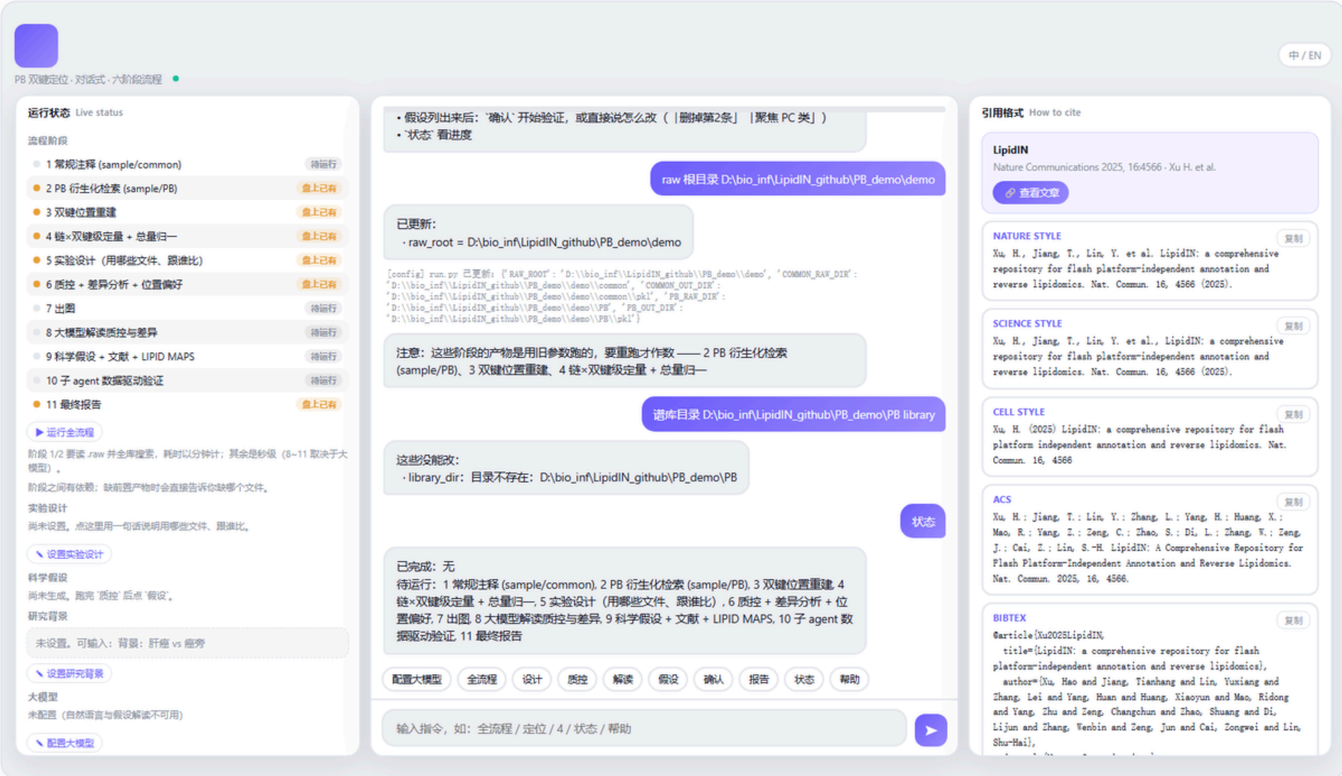
5.2 Point it at your raw data — `raw root D:\ ...`

```
raw root D:\bio_inf\LipidIN_github\PB_demo\demo
```

The agent detects `common/` and `PB/`, updates all derived paths, and the left panel refreshes. Stages whose products already exist on disk are marked **盘已有 (on disk)**.

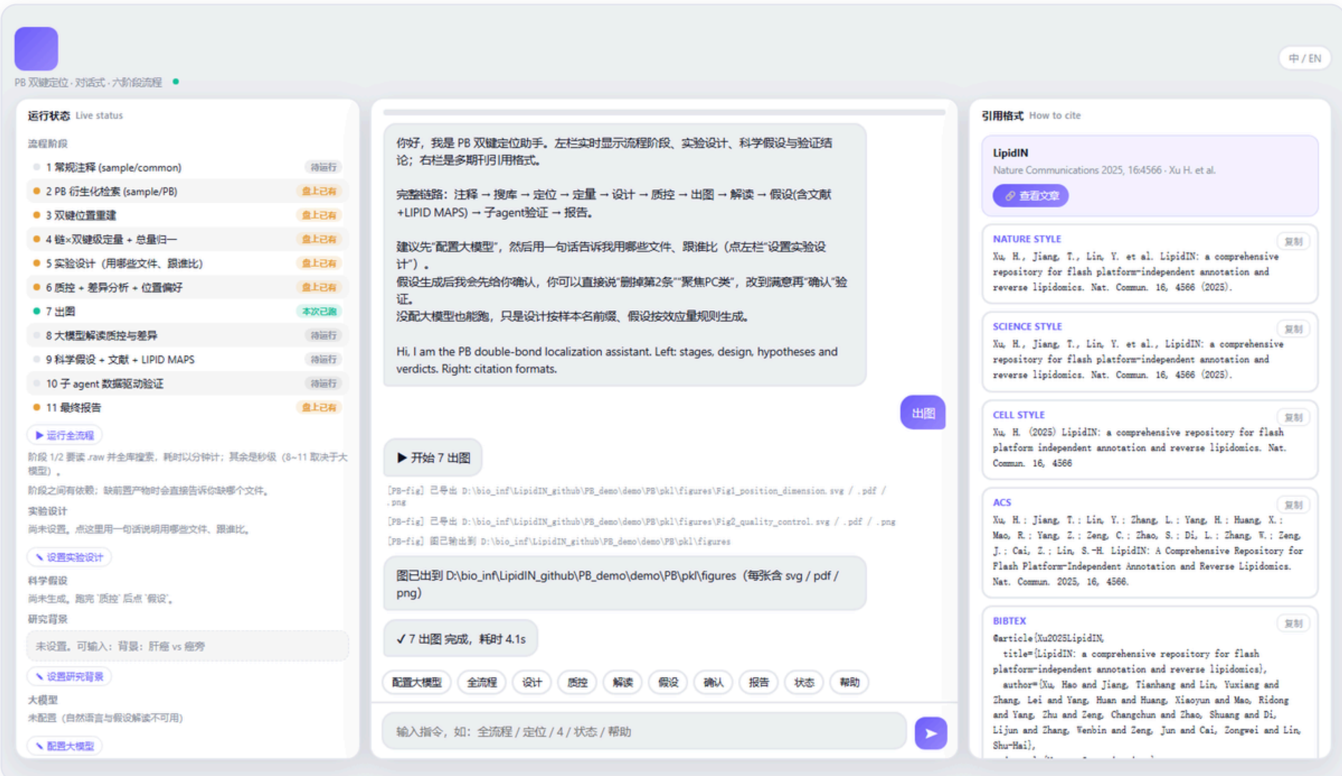


5.3 Check progress anytime — 状态 / `status`



5.4 Run the whole thing — 全流程 / `run all`

`全流程` runs Stage 1 → the hypothesis step and then **pauses for your confirmation** before verification. Or run any single stage by name/number. Here we run **plots** (`出图`); the agent streams the log live and reports where the figures went:



5.5 (Optional) Configure an LLM and set the design

- `配置大模型` / `configure LLM` — a short wizard asks for provider / key / model. Needed only for natural-language design, interpretation, and hypotheses.
- `设计` / `design`, e.g. "*use A1–A3 and B1–B3, A is the case group*".
- `背景: liver tumor vs adjacent` — gives the biology so hypotheses stay on-topic.
- After hypotheses appear: `确认` to start verification, or just say how to change them ("*drop #2*", "*focus on PC*").

---

6. What you get (outputs)

All outputs land in the `PB/pkl/` folder (and `common/pkl/` for Stage 1). The key ones:

File	Meaning
`*_PB_diagnostic.csv`	Per-peak PB diagnostic-ion hits
`*_PB_isomers.csv`	Competing double-bond isomers per species (ranked, with confidence)
`*_PB_dbpos.csv`	Finest grain: (species, f, `n-x` position) with intensity share
`PB_dbposition_quant.csv`	Quantity per (species, position) per sample (`__area` / `__norm`)
`PB_species_quant.csv`	Species-level totals (conventional layer)
`PB_species_diff.csv` / `PB_dbposition_diff.csv`	Differential analysis (species vs position layers)
`PB_qc.csv`, `PB_qc_replicate.csv`	QC + two-injection reproducibility
`PB_omega_index.csv`, `PB_position_profile.csv`, `PB_position_preference.csv`	Omega-family shares & dominant-position flips
`figures/Fig1..Fig2` (svg/pdf/png)	Publication figures
`agent_report/PB_agent_report.md`	Final Markdown report

Real demo figures produced by the `出图` step:

Fig 1 — position dimension (dominant-position flips, position-vs-amount decoupling, Δshare heatmap):

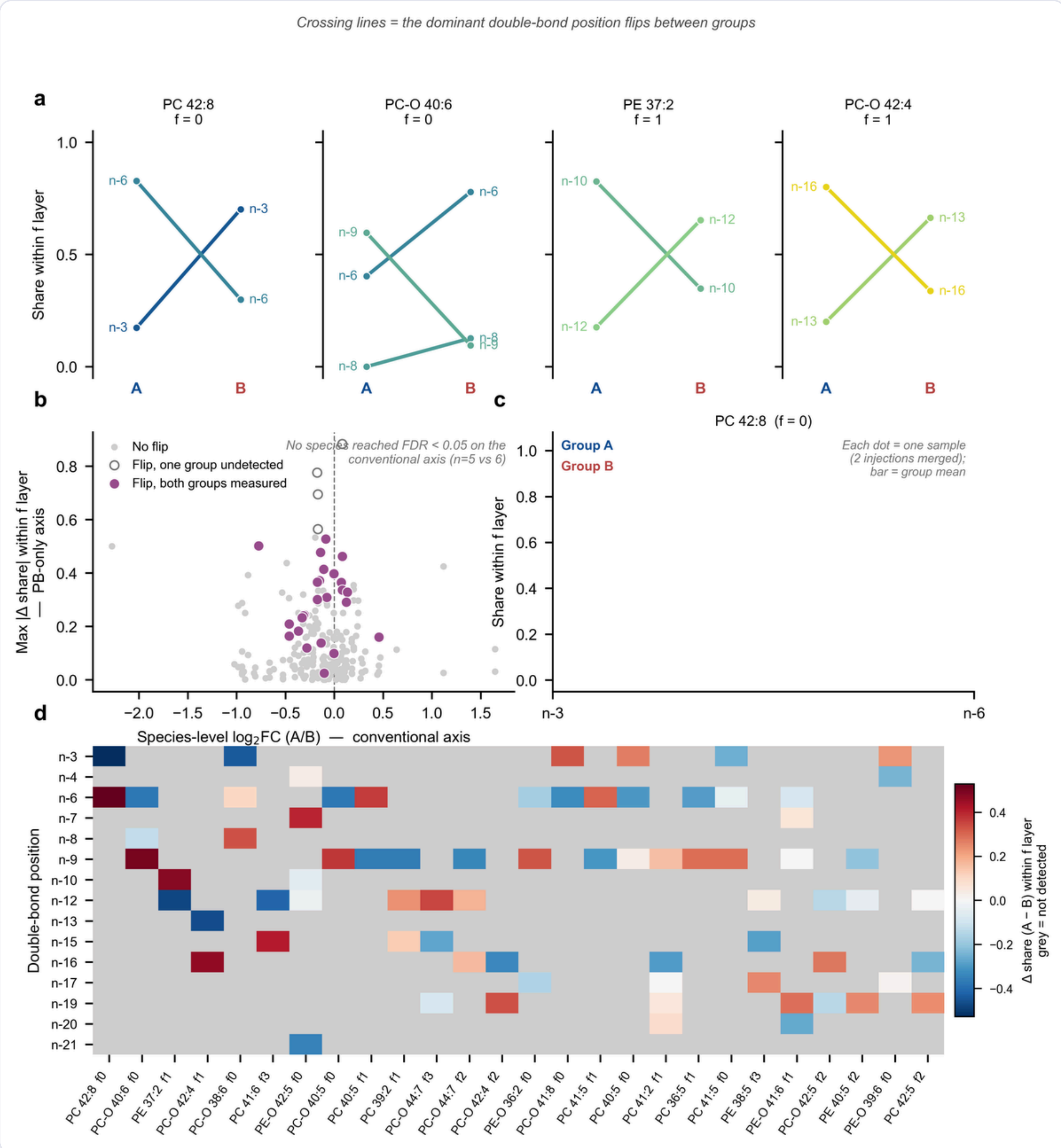
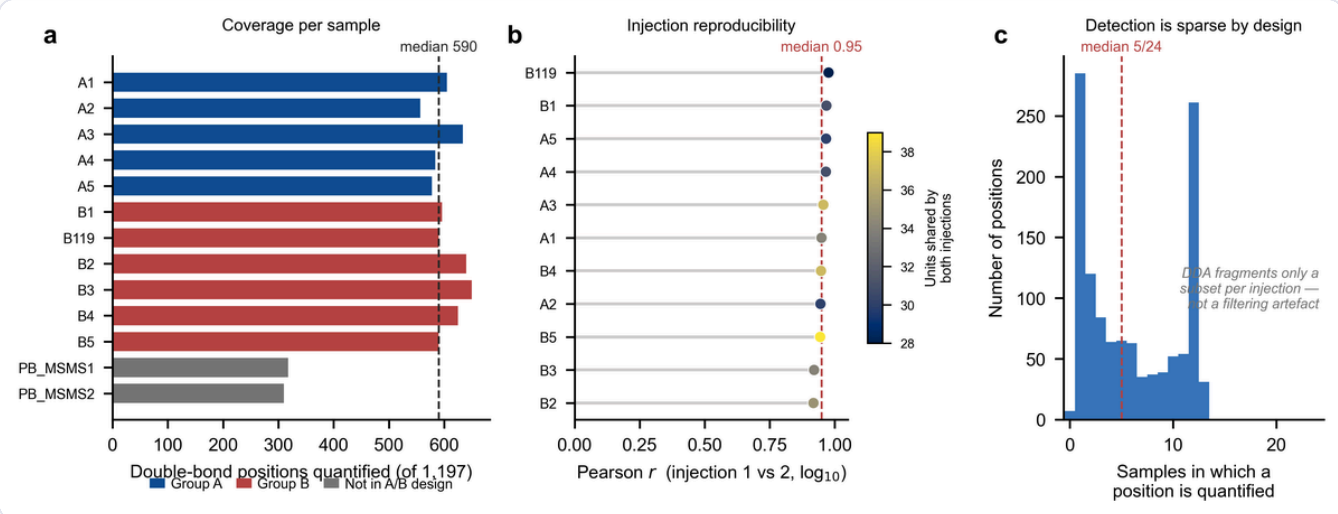


Fig 2 — quality control (coverage, injection reproducibility):



> Note: In the demo above the experiment design (case/control groups) was not set, so the per-sample dot panel (Fig 1c) is intentionally blank. Run `设计` first to populate it.

---

7. Troubleshooting (anticipated errors)

① `ImportError: eq3_search` / `eq3_similarity` **(C++ engine won't load).**
The engines are prebuilt for **CPython 3.11, Windows x64** (`eq3_search.cp311-win_amd64.pyd` , `eq3_similarity.cp311-win_amd64.pyd`). If your interpreter is a different Python version or platform, the import fails. Fix: run with **Python 3.11 on Windows x64** (e.g. `D:\anaconda3\python.exe` if it is 3.11). `eq3_similarity` can be rebuilt from source (`build_cpp.py` / `compile_cpp_similarity.py` , needs MSVC + pybind11), but `eq3_search` has **no source in this folder** (only the `.pyd`), so it cannot be recompiled here — you must keep the shipped `.pyd` and match Python 3.11.

② **A path with a space is truncated.** When you type a folder path into the chat, the agent stops the path at the first space. `D:\ … \PB library` becomes `D:\ … \PB` and you get "*directory does not exist*". Fix: put libraries/data in a **space-free path**, or set them by editing `run.py` (which handles spaces), or pass the library file paths individually where the space is inside the last segment only if unavoidable.

③ `MemoryError` **/ machine swaps during Stage 2.** The libraries are large. Keep `EQ3_WORKERS = 1` and `raw_workers = 6` (the defaults) on 16 GB; do not raise them. Close other memory-heavy apps.

④ **"Not a DDA file" / acquisition rejected.** `tm_raw2pkl` accepts **strict DDA only**. DIA/SWATH/PASEF/AllIon files are refused by design. Re-acquire in DDA, or convert appropriately.

⑤ **"common master table not found" when starting Stage 2/3.** Stage 2 needs Stage 1's `master_table.csv` , and Stage 3 needs `master_table_quant.csv` . Run Stage 1 (`annotation`) first, with quantification enabled.

⑥ **Library file not found.** The `.pkl` libraries are **not** shipped in the repo (too large). Download them from **https://zenodo.org/records/21421866** and point the agent (`谱库目录 …` , space-free) or `run.py` at that folder.

⑦ **No double-bond positions in the output.** This usually means Stage 1 (common) was skipped or the PB tolerances were changed. Keep the PB defaults (loose precursor `pb_ppm_err1=60` / `pb_da_err1=0.05`, tight fragment `pb_da_err2=0.02`) — these are deliberately set for PB diagnostic ions; tightening the precursor tolerance discards real annotations.

⑧ **Port already in use.** Another process holds `8790`. Start with `--port 8791` (any free port).

⑨ **Garbled console output / `UnicodeEncodeError`.** The entry points force UTF-8 stdout; if you invoke a module directly on a GBK console you may see mojibake — run through `run.py` / `pb_agent_server.py`, or set `PYTHONIOENCODING=utf-8`.

⑩ **LLM features do nothing.** Without `配置大模型`, free-form design/interpretation/hypotheses fall back to offline rules (still functional, less tailored). Configure a provider + key to enable them.

---

8. Citation

LipidIN — *Nature Communications* 2025, 16:4566 (Xu, H. et al.). Spectral libraries: https://zenodo.org/records/21421866